

复杂度

递归&分治

贪心

前缀和&差分

二分

复杂度

$$\text{主定理: } T(n) = aT(\lceil \frac{n}{b} \rceil) + O(n^d) = \begin{cases} O(n^d), & d > \log_b a \\ O(n^d \log n), & d = \log_b a \\ O(n^{\log_b a}), & d < \log_b a \end{cases}$$

递归&分治

- (1)该问题的规模小到一定程度就可以容易解决
(2)该问题具有最优子结构性质，利用该问题分解出的子问题的解可以合并为该问题的解
(3)该问题所分解出的各个子问题是互相独立的，即子问题之间不包含公共的子问题
- } ⇒考虑分治

分解→解决→合并

1. 分解原问题为结构相同的子问题
2. 分解到某个容易求解的边界后，用递归求解
3. 将子问题的解合并成原问题的解

注：如果各子问题是不独立的，则分治法要重复地解公共的子问题，也就做了许多不必要的工作。此时虽然也可用分治法，但一般用动态规划较好。

归并排序:

```
1 auto dfs = [&](auto dfs, int s, int t) -> void {
2     if(s >= t) return;
3     int mid = s + t >> 1;
4     dfs(dfs, s, mid), dfs(dfs, mid + 1, t);
5     vector<int> f(a.begin() + s, a.begin() + mid + 1), g(a.begin() + mid + 1,
6     a.begin() + t + 1);
7     for (int i = 0, j = 0, p = s; i < f.size() || j < g.size(); ) {
8         if(i < f.size() && (j >= g.size() || f[i] <= g[j])) {
9             a[p++] = f[i++];
10        } else{
11            a[p++] = g[j++];
12        }
13    }
14 }
```

快速排序:

```
1 mt19937 rd(time(0));
2 shuffle(a.begin() + 1, a.end(), rd);
3
4 auto q_sort = [&](auto q_sort, int l, int r) -> void {
5     if (l >= r) {
6         return;
```

```

7     }
8     int x = a[l + r >> 1];
9     vector<int> less, equa, grea;    //三路快排
10    for (int i = l; i <= r; i++) {
11        if (a[i] == x) {
12            equa.push_back(a[i]);
13        } else if (a[i] < x) {
14            less.push_back(a[i]);
15        } else if (a[i] > x) {
16            grea.push_back(a[i]);
17        }
18    }
19    int sz1 = less.size(), sz2 = equa.size();    //这里一定要建立sz, 否则后面清空less的
    时候递归时less.size()有问题
20    copy(less.begin(), less.end(), a.begin() + l);
21    copy(equa.begin(), equa.end(), a.begin() + l + sz1);
22    copy(grea.begin(), grea.end(), a.begin() + l + sz1 + sz2);
23    less = equa = grea = vector<int>();
24    q_sort(q_sort, l, l + sz1 - 1), q_sort(q_sort, l + sz1 + sz2, r);
25 };

```

贪心

排序解法： 使用邻项交换法，考虑交换顺序后是否更优

后悔解法： 先接受当前，与下一个比较，若不是最优，反悔，舍弃

前缀和&差分

前缀和： 快速查询一段区间或面积的和

差分： 快速增加或减少一段区间或面积的值

一维前缀和&差分：

```

1  for (int i = 1; i <= n; i++) {
2      a[i] += a[i - 1]; // 前缀和
3  }
4  ans = sum[j] - sum[i - 1]; // 对(i,j)查询和
5  for (int i = n; i; i--) {
6      a[i] -= a[i - 1]; // 差分
7  }
8  a[i] += k, a[j + 1] -= k; // 对(i,j)修改(+k)

```

二维/多维前缀和&差分： 利用容斥原理

```

1  for (int i = 1; i <= n; i++) {
2      for (int j = 1; j <= m; j++) {
3          a[i][j] += a[i - 1][j] + a[i][j - 1] - a[i - 1][j - 1]; // 前缀和
4      }
5  }
6  // 查询以(x1,y1)左上角,以(x2,y2)为右下角形成的矩形面积
7  ans = sum[x2][y2] - sum[x1 - 1][y2] - sum[x2][y1 - 1] + sum[x1 - 1][y1 - 1];
8  for (int i = n; i; i--) {
9      for (int j = m; j; j--) {
10         a[i][j] -= a[i - 1][j] + a[i][j - 1] - a[i - 1][j - 1]; // 差分

```

```

11     }
12 }
13 // 对以(x1,y1)为左上角,以(x2,y2)为右下角形成的矩形修改(+k)
14 a[x1][y1] += k, a[x1][y2 + 1] -= k, a[x2 + 1][y1] -= k, a[x2 + 1][y2 + 1] += k;

```

DP计算高维前缀和： 设高维空间 U 共有 D 维，需要对 $f[\cdot]$ 求高维前缀和 $pre[\cdot]$ ，令 $sum[i][state]$ 表示同 $state$ 后 $D - i$ 维相同的所有点对于 $state$ 点高维前缀和的贡献。（ $state$ 可视作 D 元组），

$sum[0][state] = f[state], pre[state] = sum[D][state]$ ，递推式为

$sum[i][state] = sum[i - 1][state] + sum[i][state']$ ，（ $state'$ 为第 i 维恰好比 $state$ 少 1 的点）

时间复杂度： $O(D \times |U|)$

```

1  for state // 含滚动
2      sum[state] = f[state]
3  for i = 0 to D
4      for state' in lexicographical order
5          sum[state] = sum[state] + sum[state']

```

树上前缀和&差分： 设 sum_i 表示结点 i 到根节点的权值之和，设 d_i 表示从叶子节点的差分数组（注意：树上前缀和差分顺序是颠倒的）

- **点前缀和：** x, y 路径上的和为 $sum_x + sum_y - sum_{lca} - sum_{fa_{lca}}$
- **边前缀和：** x, y 路径上的和为 $sum_x + sum_y - 2 \times sum_{lca}$
- **点差分：** x, y 路径上加 $k \Leftrightarrow d_x + = k, d_y + = k, d_{lca} - = k, d_{fa_{lca}} - = k$
- **边差分：** x, y 路径上加 $k \Leftrightarrow d_x + = k, d_y + = k, d_{lca} - = 2 * k$

二分

有序，满足二段性 $\Rightarrow$ 考虑二分

STL有 `lower_bound`, `upper_bound`

二分复杂度 $O(\log n)$ ，记 $check$ 函数复杂度为 $O(t)$ ，总复杂度为 $O(\log n \times t)$

整数二分：

```

1  auto chk = [&](int mid){
2      return true;
3      return false;
4  };
5  // 找左端点
6  while (l < r) {
7      int mid = l + r + 1 >> 1;
8      if (chk(mid)) {
9          l = mid;
10     } else {
11         r = mid - 1;
12     }
13 }
14 // 找右端点
15 while (l < r) {
16     int mid = l + r >> 1;
17     if (chk(mid)) {
18         r = mid;
19     } else {
20         l = mid + 1;

```

```
21     }
22 }
```

带精度二分:

```
1  auto chk = [&](double mid) {
2      return true;
3      return false;
4  };
5  for (int i = 0; i < 200; i++) {
6      double mid = (l + r) / 2;
7      if (chk(mid)) {
8          l = mid;
9      } else {
10         r = mid;
11     }
12 }
```

精度三分:

```
1  for (int i = 0; i < 200; i++) {
2      double lmid = l + (r - l) / 3, rmid = r - (r - l) / 3;
3      if (chk(lmid) < chk(rmid)) { // 此处为求最大值，舍弃小的
4          l = lmid;
5      } else {
6          r = rmid;
7      }
8  }
```

整数三分:

```
1  while (l < r) {
2      int mid = l + r >> 1;
3      if (chk(mid) < chk(mid + 1)) l = mid + 1; // 小于求最大值
4      else r = mid;
5  }
```